RITAL

Information retrieval and natural language processing
Recherche d'information et traitement automatique de la langue

Master 1 DAC, semestre 2

Laure Soulier - slides from V. Guigue and N. Thome

SCIENCES SORBONNE UNIVERSITÉ

ISIR INSTITUT DES SYSTÈMES INTELLIGENTS ET DE ROBOTIQUE

MLIA Machine Learning & Deep Learning for Information Access

Text representations

Outline

SCIENCES SORBONNE UNIVERSITÉ

1. Preprocessing

- encoding (latin, utf8, ...)
- ponctuation
- stemming
- lemmatization
- tokenization
- capitals/lower case
- regex
- ...

2. BoW Model

- Dictionary
- Vectorial format
- TF-IDF
- Binary/non-binary
- N-grams

3. Learning

- Doc / sentence / paragraph classification
- Linear/non-linear classifiers?
- Naive Bayes, logistic regression, SVM

4. Hyper-parameter optimization

Trade-off between:

- Accurate BoW representation (expressive vocab, N-gram, etc)
- Fighting overfitting: limit dimension explosion

Pre-processing: encoding

`http://sametmax.com/lencoding-en-python-une-bonne-fois-pour-toute/`

- Sur le disque, les fichiers sont encodés de manière spécifique...
- En python, les strings sont encodées de manière spécifique...

`http://sametmax.com/lencoding-en-python-une-bonne-fois-pour-toute/`

- Sur le disque, les fichiers sont encodés de manière spécifique...
- En python, les strings sont encodées de manière spécifique...
- L'ouverture des fichiers est souvent associée à un encodage !!!

$\Rightarrow$ Comment gérer cela?

http://sametmax.com/lencoding-en-python-une-bonne-fois-pour-toute/

- Sur le disque, les fichiers sont encodés de manière spécifique...
- En python, les strings sont encodées de manière spécifique...
- L'ouverture des fichiers est souvent associée à un encodage !!!

$\Rightarrow$ Comment gérer cela?

Solution 1

1. Ouverture en binaire des fichiers (e.g. en python)
2. Conversion des strings depuis un encodage connu
   `str.decode('utf8')`
   `unicodedata`, `unidecode`

Solution 2

1. Vérifier le type d'encodage + convertir avant

What is textual data?

- A **series** of letters

```
t h e _ c a t _ i s ...
```

- A **series** of words

```
the cat is ...
```

- A **set** of words

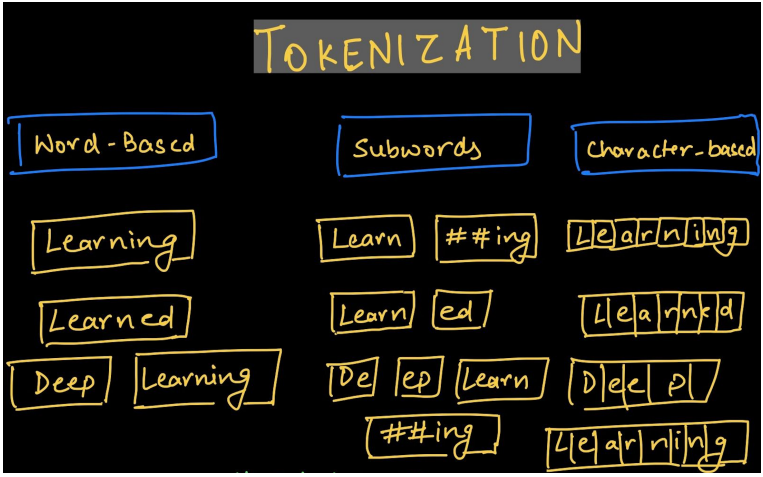
in alphabetical order

```
cat
is
the
...
```

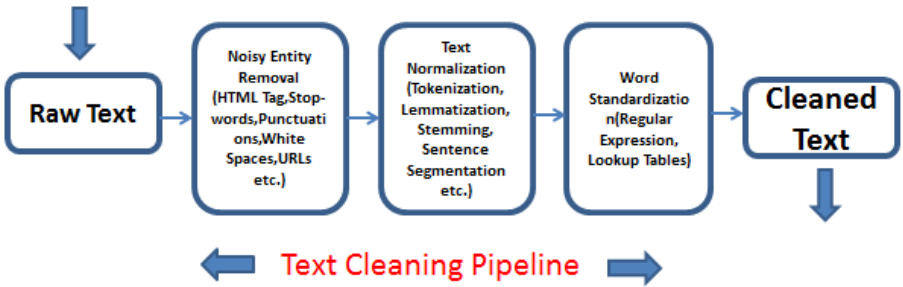
- N-gram dictionary:

```
ex bi-grams: BEG_the the_cat cat_is is_...
```

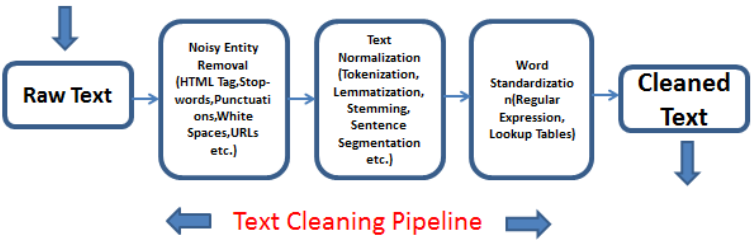
- Text: extracts "tokens", *i.e.* raw inputs
- Which tokens, *e.g.* characters, words, N-grams, sentences?



- Some input token: noise to the classification tasks.
  - Noise: task-dependent, and on the training dataset size / robustness to overfitting



- Ponctuation, capitals/lower case: remove or keep it?
- Stop word: empty meaning word, *e.g.* "the"
  - Use pre-defined "black list" (nltk) or upper frequency bound on target corpus
- Removing rare words (occurring less than a threshold)



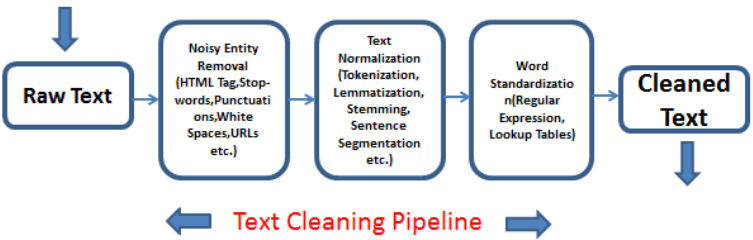
Lemmatization:

in linguistics is the process of grouping together the inflected forms of a word so they can be analysed as a single item, identified by the word's lemma, or dictionary form

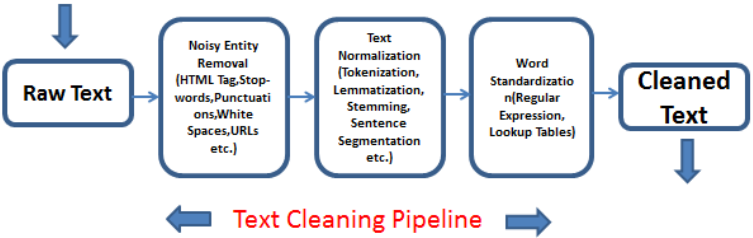
$\Rightarrow$ Requires advanced linguistic ressources including words and inflected forms.
E.g. : lions $\Rightarrow$ lion; are $\Rightarrow$ be; ...

Stemming:

A statistical approximated process of the lemmatization
E.g. : removing **s** or **ly** at the end of the words



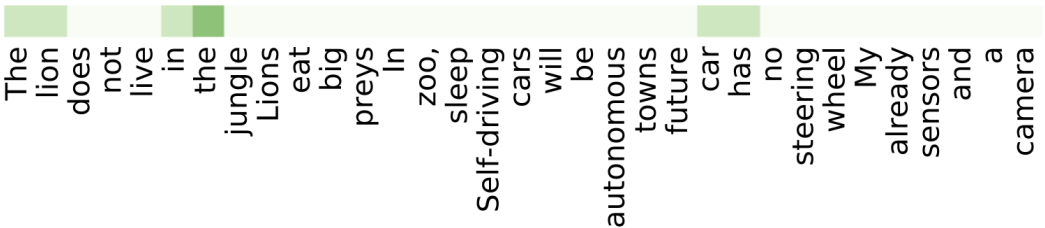
- Regular expression, *e.g.* for removing "." or expanding words' contractions ("I'll" $\rightarrow$ "I will")



Dimensionality (and noise) reduction

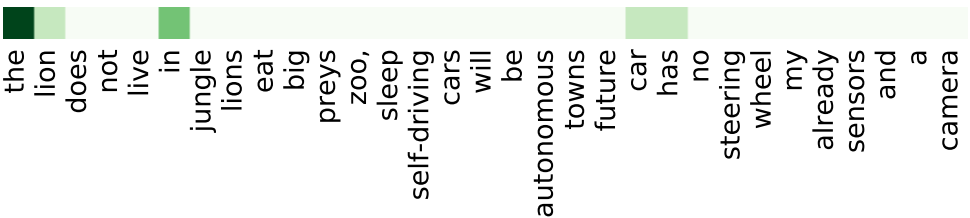
```
documents = ['The lion does not live in the jungle',\
'Lions eat big preys',\
'In the zoo, the lion sleep',\
'Self-driving cars will be autonomous in towns',\
'The future car has no steering wheel',\
'My car already has sensors and a camera']
```

Original dictionary:



```
documents = ['The lion does not live in the jungle',\
'Lions eat big preys',\
'In the zoo, the lion sleep',\
'Self-driving cars will be autonomous in towns',\
'The future car has no steering wheel',\
'My car already has sensors and a camera']
```

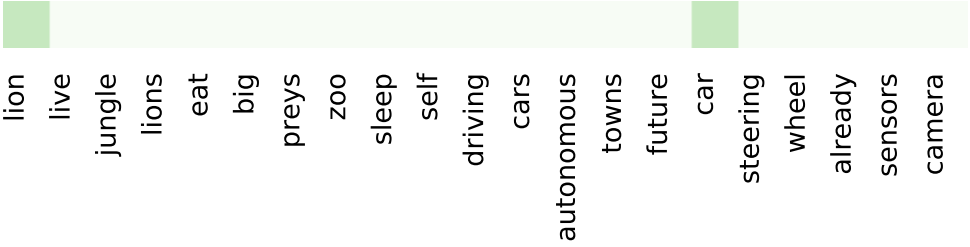
Removing capitals:



Dimensionality (and noise) reduction

```
documents = ['The lion does not live in the jungle',\
'Lions eat big preys',\
'In the zoo, the lion sleep',\
'Self-driving cars will be autonomous in towns',\
'The future car has no steering wheel',\
'My car already has sensors and a camera']
```

Removing stop words:



Implementation: black list (nltk) or upper frequency bound

```
documents = ['The lion does not live in the jungle',\
'Lions eat big preys',\
'In the zoo, the lion sleep',\
'Self-driving cars will be autonomous in towns',\
'The future car has no steering wheel',\
'My car already has sensors and a camera']
```

Removing rare words occurring less than a threshold :

Dictionary = {lion, car}

$\Rightarrow$ too extreme in this toy example...

But a good idea in real situations.

Remainder: rare words represent a large part of the dictionary

$\Rightarrow$ Tricky setting of the thresholds (upper & lower bounds)

```
documents = ['The lion does not live in the jungle',\
'Lions eat big preys',\
'In the zoo, the lion sleep',\
'Self-driving cars will be autonomous in towns',\
'The future car has no steering wheel',\
'My car already has sensors and a camera']
```

Lemmatization:

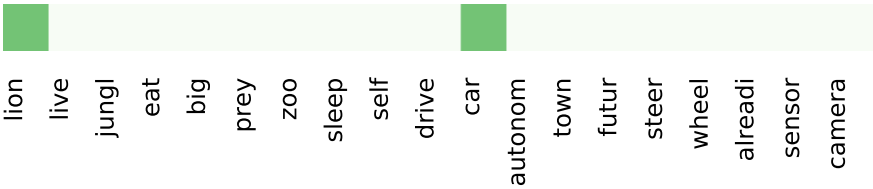
in linguistics is the process of grouping together the inflected forms of a word so they can be analysed as a single item, identified by the word's lemma, or dictionary form

$\Rightarrow$ Requires advanced linguistic ressources including words and inflected forms.
E.g. : lions $\Rightarrow$ lion; are $\Rightarrow$ be; ...

```
documents = ['The lion does not live in the jungle',\
'Lions eat big preys',\
'In the zoo, the lion sleep',\
'Self-driving cars will be autonomous in towns',\
'The future car has no steering wheel',\
'My car already has sensors and a camera']
```

Stemming:

A statistical approximated process of the lemmatization
E.g. : removing **s** or **ly** at the end of the words

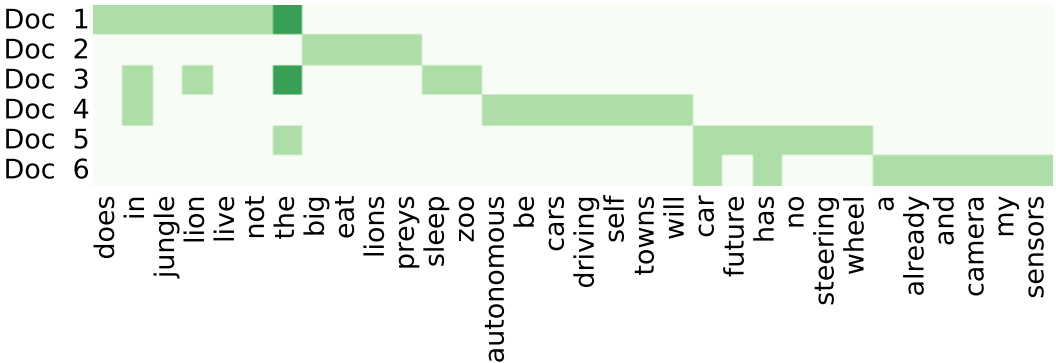


Note: it is not a problem to create invalid words... If they are stable!

Corpus representation

```
documents = ['The lion does not live in the jungle',\
'Lions eat big preys',\
'In the zoo, the lion sleep',\
'Self-driving cars will be autonomous in towns',\
'The future car has no steering wheel',\
'My car already has sensors and a camera']
```

Corpus mapping on the basic dictionary:



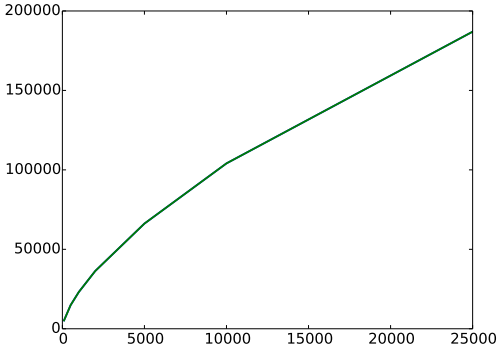
Vocabulary size:

$$|V| \propto \log(N), \qquad N = \text{number of documents}$$

On movie reviews :

$|V|$ with respect to # reviews
Let's have a closer look on the axes !!!

25k docs $\Leftrightarrow$ 200k words !!

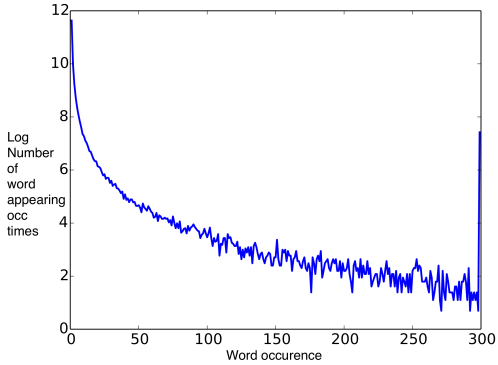


Vocabulary size:

$$|V| \propto \log(N), \qquad N = \text{number of documents}$$

Word occurence distribution:

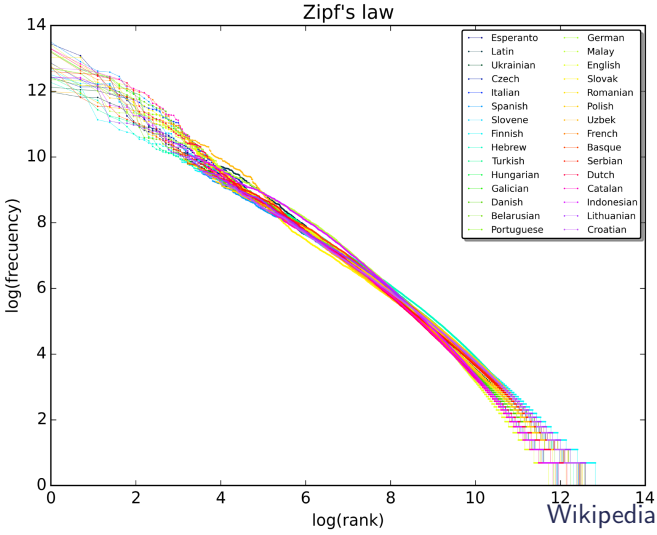
$Occ_i = \{w |\ occurence(w) = i\}$
Plot $= \log(|Occ_i|)$ wrt i



Zipf law in several languages

$$f(n) = \frac{k}{n}$$

- n # documents
- k constant



Wikipedia statistics

- Simplest encoding of tokens: **one-hot representation**
- Binary vector of vocabulary size $|V|$, with 1 corresponding to term index (0 otherwise)
- $|V|$ small for chars (~ 10), large for words ($\sim 10^4$), huge for N-grams / sentences
- Basis for constructing Bag of Word (BoW) Models

the dog is on the table

Sentence structure = costly handling ⇒ Elimination !

Bag of words representation

1. Extraction of vocabulary V
2. Each document becomes a counting vector : $d \in \mathbb{N}^{|V|}$

this new iPhone, what a marvel

An iPhone? What a scam!

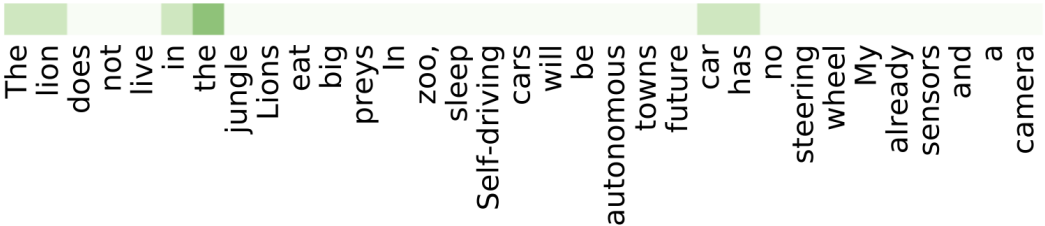
marvel	this	new	iPhone	what	an	scam
1	1	1	1	1	0	0
0	0	0	1	1	1	1

Note: d is always a sparse vectors, mainly composed of 0

Set of toy documents:

```
documents = ['The lion does not live in the jungle',\
'Lions eat big preys',\
'In the zoo, the lion sleep',\
'Self-driving cars will be autonomous in towns',\
'The future car has no steering wheel',\
'My car already has sensors and a camera']
```

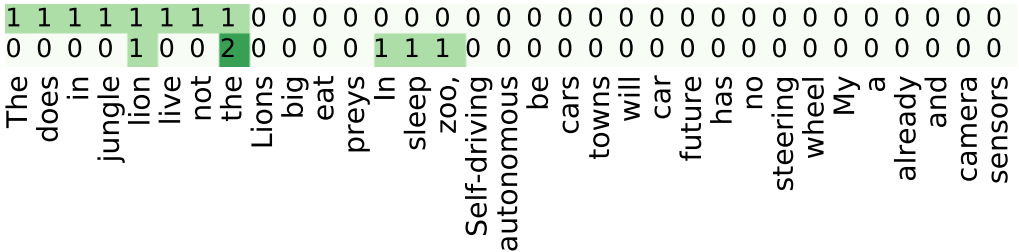
Dictionary



Green level $\propto$ nb occurences

Counting words appearing in 2 documents:

The lion does not live in the jungle
In the zoo, the lion sleep



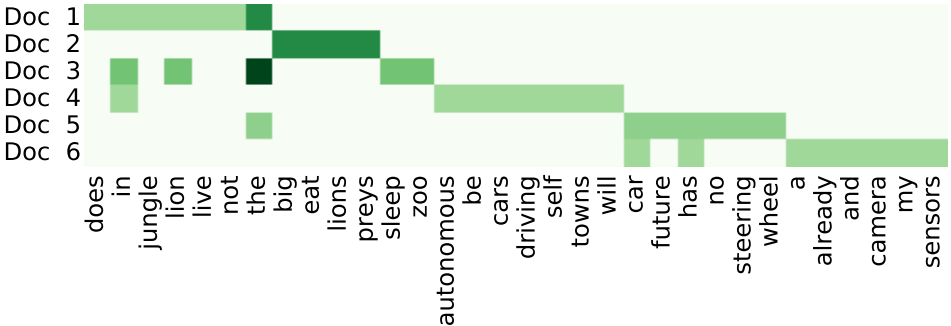
\+ We are able to vectorize textual information

− Dictionary requires preprocessing

Classical issues in text modeling

Every document in the corpus has the **same nearest neighbor**...

A strong magnet with many words (=long document)



Normalization $\Rightarrow$ descriptors = **term frequency** in the document

$$\forall i, \qquad \sum_k d_{ik}^{(tf)} = 1$$

Frequent word are overweighted...

if word k is in most documents, it is probably useless

Introduction of the **document frequency**:

$$\mathrm{df_k} = \frac{|\{\mathrm{d} : t_k \in \mathrm{d}\}|}{|C|}, \qquad \text{Corpus : } C = \{\mathrm{d}_1, \ldots, \mathrm{d}_{|C|}\}$$

Tf-idf coding = term frequency, inverse document frequency:

$$d_{ik}^{(tfidf)} = d_{ik}^{(tf)} \log \frac{|C|}{|\{\mathrm{d} : t_k \in \mathrm{d}\}|}$$

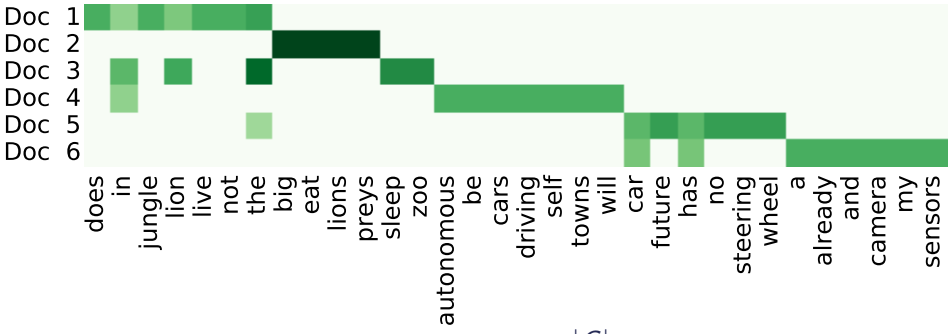
$\Rightarrow$ A strong idea to focus on **keywords**...

... But not always strong enough to get rid of all stop words

$\Rightarrow$ TFIDF could be combined with blacklists (see next course)

Frequent word are overweighted...

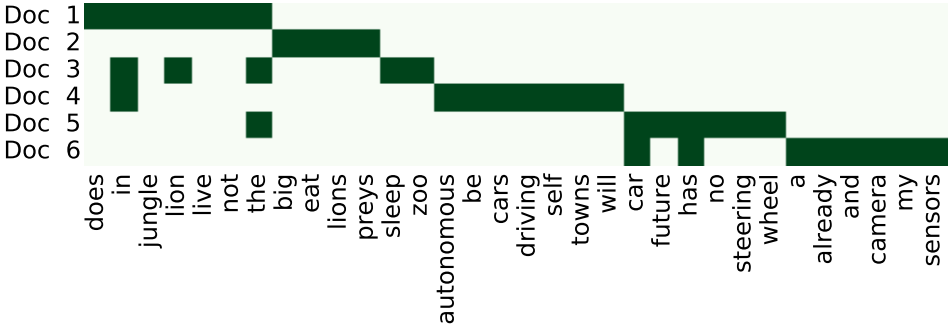
if word k is in most documents, it is not discriminant



$$d_{ik}^{(tfidf)} = d_{ik}^{(tf)} \log \frac{|C|}{|\{\mathrm{d} : t_k \in \mathrm{d}\}|}$$

In some specific cases (e.g. sentiment classification), it is more robust to remove all information about frequency...

$\Rightarrow$ Presence coding



$$d_{ik}^{(pres)} = \begin{cases} 1 & \text{if word k is in } d_i \\ 0 & \text{else} \end{cases}$$

+ Strengths
  - Easy, light, fast (Real-time systems, IR (indexing)...)
  - Opportunity to enrich (POS, context encoding,..)
  - Efficient implementations `nltk, sklearn`
  - Still very effective on document classification see project

− Drawbacks
  - Loose document/sentence structure
    - Can be mitigated with N-grams
  - ⇒ **BUT: several tasks almost impossible to tackle**
    - NER, POS tagging, SRL
    - Text generation

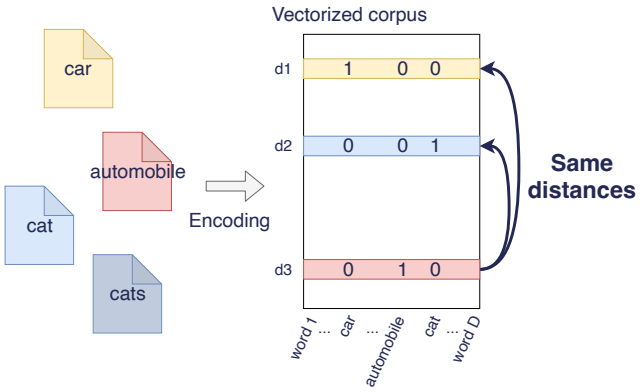
All words are orthogonal:

Considering virtual 2 documents made of a single word :

$$\begin{bmatrix} 0 & \dots & 0 & d_{ik} > 0 & \dots & 0 \\ 0 & \dots & d_{jk'} > 0 & 0 & \dots & 0 \end{bmatrix}$$

Then: $k \neq k' \Rightarrow \mathrm{d}_i \cdot \mathrm{d}_j = 0$

...even if $w_k = \textit{lion}$ and $w_{k'} = \textit{lions}$



⇒ Definition of the **semantic gap**

No metrics between words

- Syntactic difference ⇒ orthogonality of the representation vectors
- Word groups : more intrinsic semantics... ... but fewer match with other document
  - N-grams ⇒ dictionary size ↗
  - N-grams = great potential... but require careful preprocessings

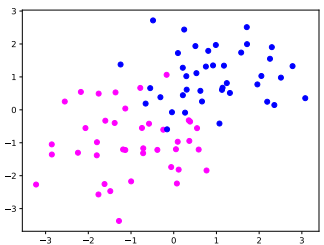
This film was not interesting

- Unigrams: `this`, `film`, `was`, `not`, `interesting`
- bigrams: `this_film`, `film_was`, `was_not`, `not_interesting`
- N-grams... + combination: e.g. 1-3 grams

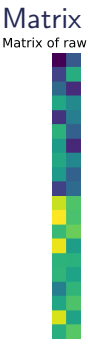
⇒ **See project**

A classical toy example to illustrate the curse of dimensionality:

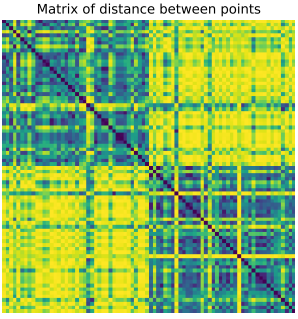
Original dataset:



Matrix view



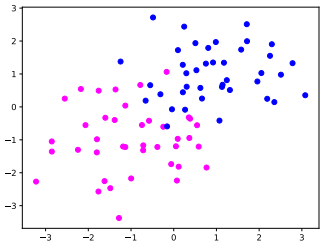
Distance matrix



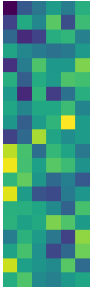
Easy problem / classes are clearly separated

A classical toy example to illustrate the curse of dimensionality:

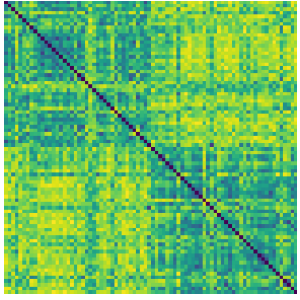
Original dataset:



Matrix view



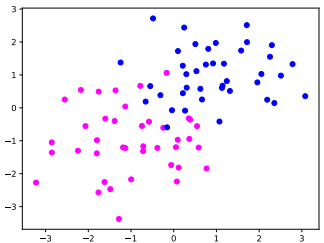
Distance matrix



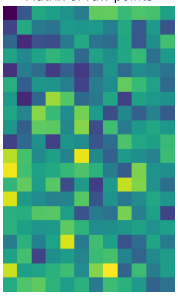
Adding some noisy dimensions in the dataset

A classical toy example to illustrate the curse of dimensionality:

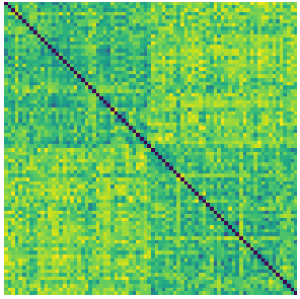
Original dataset:



Matrix view



Distance matrix

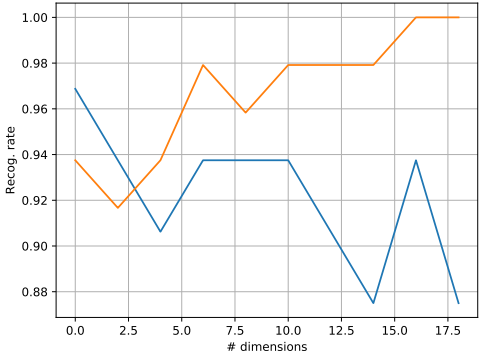


Adding **more** noisy dimensions in the dataset

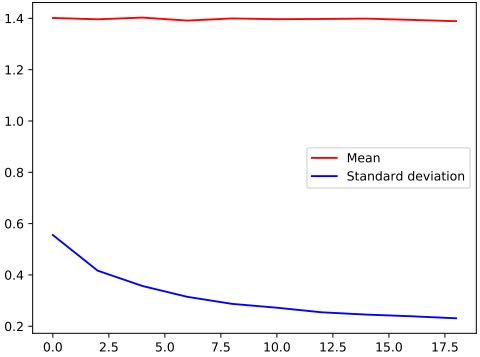
$\Rightarrow$ Euclidian distance is very sensitive to the dimensionality issue

A classical toy example to illustrate the curse of dimensionality:

Basic classifier on those datasets



Distances between points in the dataset



$\Rightarrow$ Learn accuracy $\nearrow$, test accuracy $\searrow$ = **overfitting**

$\Rightarrow$ All points tend to lay on an hypersphere (they become equidistant)

Given documents $\mathrm{d}_i \in \mathbb{R}^{|D|}$ and $d_j \in \mathbb{R}^{|D|}$ with the dictionary D.

First idea : **Euclidian metrics**

$$d(\mathrm{d}_i, \mathrm{d}_j) = \|\mathrm{d}_i - \mathrm{d}_j\| = \sqrt{\sum_k (d_{ik} - d_{jk})^2}$$

But:

$$d(\mathrm{d}_i, \mathrm{d}_j) = \sqrt{\|\mathrm{d}_i\|^2 + \|\mathrm{d}_j\|^2 - 2\mathrm{d}_i \cdot \mathrm{d}_j}$$

$\Rightarrow$ Sensitive to the norm of d or to the ratio $\mathrm{d}_i \cdot \mathrm{d}_j$ vs $\|\mathrm{d}\|$

- **Euclidian distance**
  $\Rightarrow$ not robust enough
- **Inner product**

$$sim(\mathrm{d}_i, \mathrm{d}_j) = \frac{\mathrm{d}_i \cdot \mathrm{d}_j}{\|\mathrm{d}_i\| \|\mathrm{d}_j\|} = \cos(\widehat{\vec{\mathrm{d}_i}, \vec{\mathrm{d}_j}}) \propto \sum_k d_{ik} d_{jk}$$

  $\Rightarrow$ focusing on common non-zeros dimensions
- **Kullback Leibler**
  Assuming that each document can be seen as a distribution over words (ie, $\forall i,\ \sum_k d_{ik} = 1$)

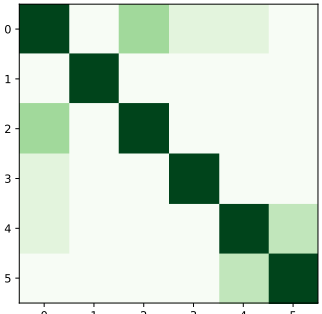
$$D_{\mathrm{KL}}(\mathrm{d}_i \| \mathrm{d}_j) = \sum_k d_{ik} \log \frac{d_{ik}}{d_{jk}}$$

  $\Rightarrow$ not very stable, take care of $d_{ik} = 0$ or $d_{jk} = 0$

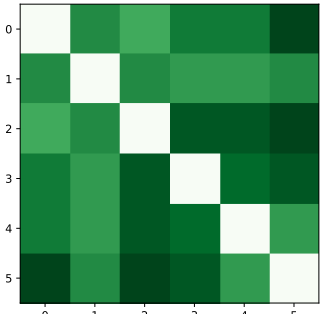
Metrics on our example

```
documents = ['The lion does not live in the jungle',\
'Lions eat big preys',\
'In the zoo, the lion sleep',\
'Self-driving cars will be autonomous in towns',\
'The future car has no steering wheel',\
'My car already has sensors and a camera']
```

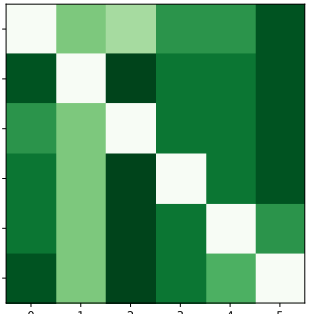
Cos



Euclidian



KL

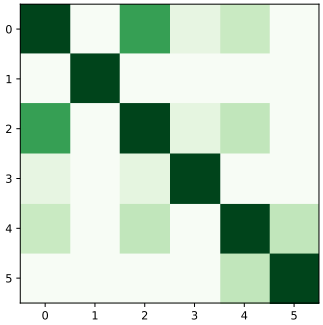


Basic representation of texts... **Too much noise !**

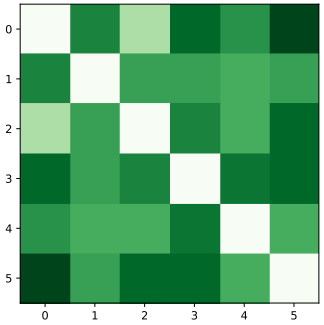
Metrics on our example

```
documents = ['The lion does not live in the jungle',\
'Lions eat big preys',\
'In the zoo, the lion sleep',\
'Self-driving cars will be autonomous in towns',\
'The future car has no steering wheel',\
'My car already has sensors and a camera']
```

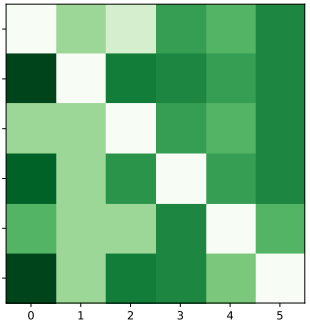
Cos



Euclidian



KL

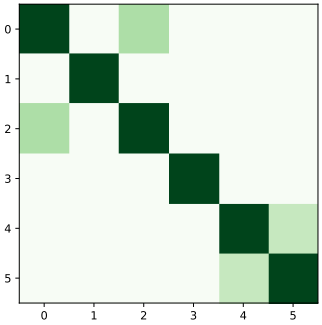


Preprocessing (removing capitals/puntuation) $\Rightarrow$ situation still confuse

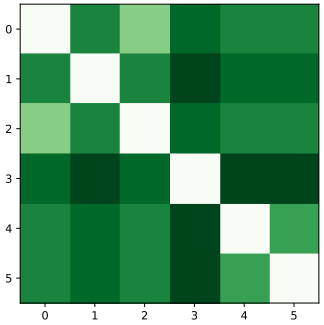
Metrics on our example

```
documents = ['The lion does not live in the jungle',\
'Lions eat big preys',\
'In the zoo, the lion sleep',\
'Self-driving cars will be autonomous in towns',\
'The future car has no steering wheel',\
'My car already has sensors and a camera']
```

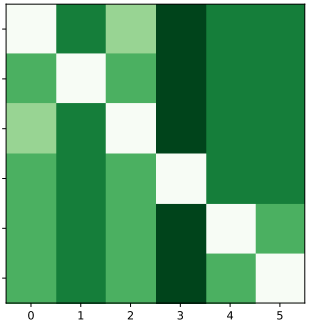
Cos



Euclidian



KL

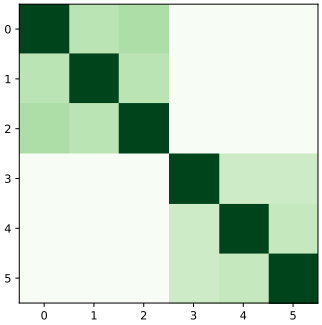


Preprocessing + removing stop words $\Rightarrow$ slight improvment

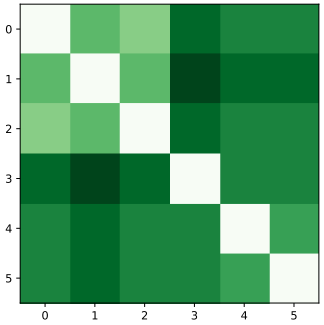
Metrics on our example

```
documents = ['The lion does not live in the jungle',\
'Lions eat big preys',\
'In the zoo, the lion sleep',\
'Self-driving cars will be autonomous in towns',\
'The future car has no steering wheel',\
'My car already has sensors and a camera']
```

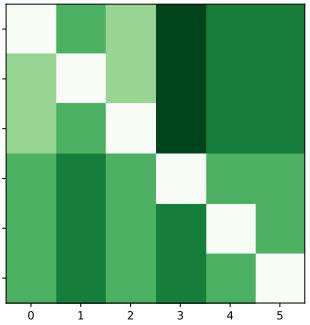
Cos



Euclidian



KL



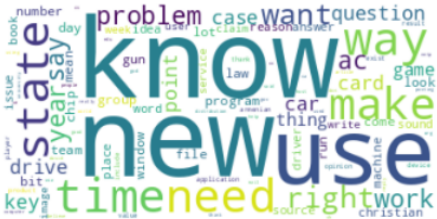
Preprocessing + removing stop words + stemming $\Rightarrow$ distinction between classes

Unsupervised approaches for text representation

Outline

- Use word clouds to visualize the most relevant words from a corpus
- From raw data (using stop words)
- From BoW (using vectorizer - and stop words)
- Ex: fetch20newsgroups (practical)

Raw data



BoW (frequencies)


alt.atheism
comp.graphics
comp.os.ms-windows.misc
comp.sys.ibm.pc.hardware
comp.sys.mac.hardware
comp.windows.x
misc.forsale
rec.autos
rec.motorcycles
rec.sport.baseball
rec.sport.hockey
sci.crypt
sci.electronics
sci.med
sci.space
soc.religion.christian

- Classes: semantic informtion
  - Odd ratios can improve discriminability (often in log):

$$S_{odds}(i) = \frac{p_i/(1-p_i)}{q_i/(1-q_i)} = \frac{p_i(1-q_i)}{q_i(1-p_i)}$$

    - p_i frequency of t_i in class 1 ($\oplus$) and q_i is frequency of t_i in class 2 ($\ominus$)
- How to extract semantic info without labels?
  $\Rightarrow$ **Unsupervised learning**

- Modeling: Word count (and BoW storage)

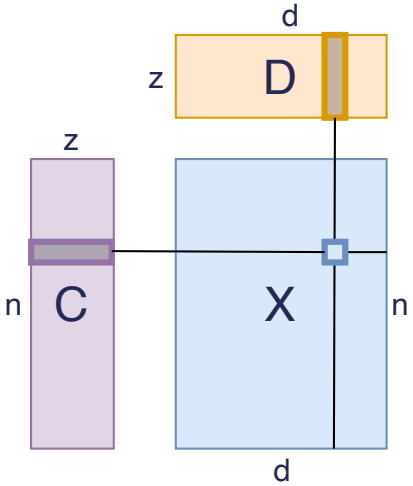
$$X = \begin{array}{c} \\ \mathbf{d}_i \rightarrow \end{array} \begin{array}{c} \mathbf{t}_j \\ \downarrow \\ \begin{pmatrix} x_{1,1} & \cdots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{N,1} & \cdots & x_{N,d} \end{pmatrix} \end{array}$$

- Basic proposal: semantics = metrics = similarity between columns in BoW

$$s(j,k) = \langle t_j, t_k \rangle, \qquad \text{Normalized: } s_n(j,k) = \cos(\theta) = \frac{\mathbf{t}_j \cdot \mathbf{t}_q}{\|\mathbf{t}_j\| \, \|\mathbf{t}_q\|}$$

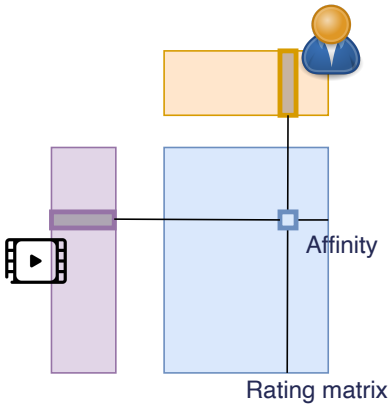
  - If two terms appear in the same document, they are similar

Matrix factorization = basic tool to understand the data



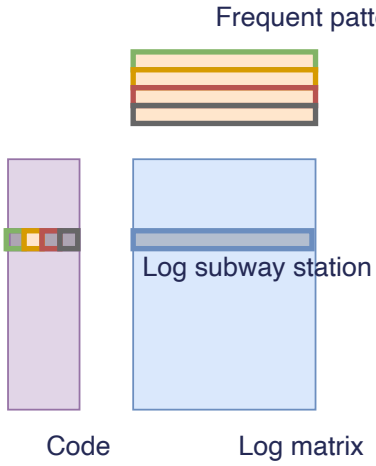
- Extract a compact representation
  - for words
  - for documents
- = focus on high-energy phenomenon
  - Eliminate noise in the data
- Optimal data compression [Mean Square criterion]

Matrix factorization = basic tool to understand the data



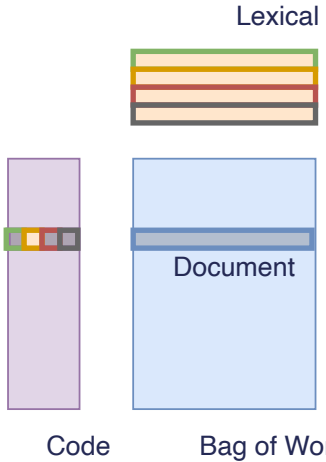
- Extract a compact representation
  - for words
  - for documents
- = focus on high-energy phenomenon
  - Eliminate noise in the data
- Optimal data compression [Mean Square criterion]

Matrix factorization = basic tool to understand the data



- Extract a compact representation
  - for words
  - for documents
- = focus on high-energy phenomenon
  - Eliminate noise in the data
- Optimal data compression [Mean Square criterion]

Matrix factorization = basic tool to understand the data



- Extract a compact representation
  - for words
  - for documents
- = focus on high-energy phenomenon
  - Eliminate noise in the data
- Optimal data compression [Mean Square criterion]

- In NLP : SVD = LSA: Latent Semantic Analysis
- Idea : grouping similar documents / learning a representation of documents

$$
\begin{array}{ccccc}
X & & = & U & \Sigma & V^T \\
\mathbf{t}_j & & & \hat{\mathbf{d}}_i & & \\
\downarrow & & & \downarrow & &
\end{array}
$$

$$
\mathbf{d}_i \rightarrow \begin{pmatrix} x_{1,1} & \dots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{N,1} & \dots & x_{N,d} \end{pmatrix} = \begin{pmatrix} (& \mathbf{u_1} &) \\ & \vdots & \\ (& \mathbf{u}_k &) \end{pmatrix} \begin{pmatrix} \sigma_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & \sigma_k \end{pmatrix} \begin{pmatrix} \begin{pmatrix} \\ \mathbf{v_1} \\ \\ \end{pmatrix} & \dots & \begin{pmatrix} \\ \mathbf{v}_k \\ \\ \end{pmatrix} \end{pmatrix}
$$

- Good news: functions well on sparse matrices
  - See `TruncatedSVD` in `sklearn.decomposition`

Factorization = robustness & clustering ability

S. Deerwester, et al., JSIS 1990
Indexing by latent semantic analysis

Selecting the k greatest singular values: rank-k approximation of the occurence matrix X

$$\underset{}{X} \quad = \quad \underset{\hat{\mathbf{d}}_i}{U} \quad \Sigma \quad V^T$$

$$\mathbf{d}_i \rightarrow \begin{pmatrix} x_{1,1} & \dots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{N,1} & \dots & x_{N,d} \end{pmatrix} = \begin{pmatrix} \mathbf{u_1} \\ \vdots \\ \mathbf{u}_k \end{pmatrix} \begin{pmatrix} \sigma_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & \sigma_k \end{pmatrix} \begin{pmatrix} \mathbf{v_1} & \dots & \mathbf{v}_k \end{pmatrix}$$

- Each $\mathbf{v}_i \in \mathbb{R}^d$: a weight vector associated to the vocabulary
- The base $\{\mathbf{v}_1, \dots, \mathbf{v}_k\}$ is orthogonal
  - Each $\mathbf{v}_i$ corresponds to a different **lexical field**
- The new $\hat{\mathbf{d}}_i$ representation $\boldsymbol{u_i}$: weight vector associated to the lexical fields
  - **Clustering:** the strongest weight gives the document class

Thomas K. Landauer, Peter W. Foltz et Darrell Laham, Discourse Processes, vol. 25, 1998
Introduction to Latent Semantic Analysis

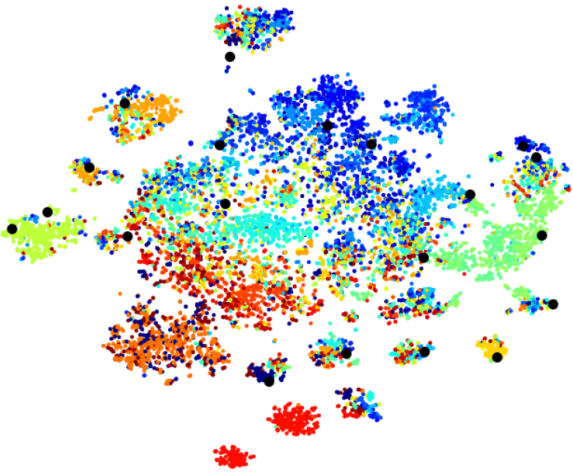
Usages:

- Clustering (each eigen vector describes a *topic)*
- Semantics: words have a representation over the topics
- IR Improvement:
  - Query expansion based on the topic definition
  - Detection of polysemic terms
- new representation $\Rightarrow$ new metrics
  - opportunities in question answering
    - Finding the part of a document relating to a specific topic
  - Automated summarization
    - Document segmentation + sentence extraction
  - TDT : Topic detection & Tracking

$$
\begin{array}{cccccc}
 & X & = & U & \Sigma & V^T \\
 & \mathbf{t}_j & & \hat{\mathbf{d}}_i & & \\
 & \downarrow & & \downarrow & & \\
\mathbf{d}_i \rightarrow & \begin{pmatrix} x_{1,1} & \dots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{N,1} & \dots & x_{N,d} \end{pmatrix} & = & \begin{pmatrix} (& \mathbf{u_1} &) \\ & \vdots & \\ (& \mathbf{u}_k &) \end{pmatrix} & \begin{pmatrix} \sigma_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & \sigma_k \end{pmatrix} & \left(\begin{pmatrix} & & \\ \mathbf{v_1} & & \end{pmatrix} \dots \begin{pmatrix} & & \\ \mathbf{v}_k & & \end{pmatrix} \right)
\end{array}
$$

- On fetch20newsgroups, test with $k = 20$
- **Qualitative assessment:**
  - $\mathbf{v}_i \in \mathbb{R}^d$: look at most important words
  - $\hat{\mathbf{d}}_i := \mathbf{u}_i$: cluster each document / topics
  - Word clouds for each cluster
  - t-SNE after LSA projection

- Each dot: cluster center
- color code: GT classes



$$
\begin{array}{cccccc}
 & X & = & U & \Sigma & V^T \\
 & \mathbf{t}_j & & \hat{\mathbf{d}}_i & & \\
 & \downarrow & & \downarrow & & \\
\mathbf{d}_i \rightarrow & \begin{pmatrix} x_{1,1} & \dots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{N,1} & \dots & x_{N,d} \end{pmatrix} & = & \begin{pmatrix} (& \mathbf{u_1} &) \\ & \vdots & \\ (& \mathbf{u}_k &) \end{pmatrix} & \begin{pmatrix} \sigma_1 & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & \sigma_k \end{pmatrix} & \begin{pmatrix} \begin{pmatrix} & \\ & \mathbf{v_1} \\ & \end{pmatrix} & \dots & \begin{pmatrix} & \\ & \mathbf{v}_k \\ & \end{pmatrix} \end{pmatrix}
\end{array}
$$

- On fetch20newsgroups, test with $k = 20$
- **Quantitative assessment:** with 3 metrics
  - Purity: $p = \frac{|y^*|}{|C|}$, where y^* is the most frequent (GT) label in cluster C
  - Rand score: `https://en.wikipedia.org/wiki/Rand_index`
  - Adjusted Rand score (ARS)

- Fully based on BOW: no word dependency modeling
- Linear model
- Topic modeling linked to a corpus
  - problem with rare words in small corpus
  - bias of the corpus

- Still a BOW modeling

$$X = \begin{matrix} & \\ \mathbf{d}_i \rightarrow & \end{matrix} \begin{matrix} \mathbf{t}_j \\ \downarrow \end{matrix} \begin{pmatrix} x_{1,1} & \cdots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{N,1} & \cdots & x_{N,d} \end{pmatrix}$$

- Algorithm that scale up well
  - Possible **on-line** version of the algorithm
  - Can be linked to chinese restaurant / indian buffet process
    - $\Rightarrow$ Discover k in an online process
- Orthogonality is not longer enforced

New vision of k-means :

- k clusters
- A priori probabilities : $\pi_k = p(\theta_k)$
- Probability of a word in a cluster : $p(w_j|\theta_k) = \mathbb{E}_{d \in \mathcal{D}_k}[w_j]$
- Document hard assignment in a cluster: $p(\theta_k|d_i) = 1/0$

$$y_i = \arg\max_k p(\theta_k)p(d_i|\theta_k) = \arg\max_k \log(\pi_k) + \sum_{w_j \in d_i} \log p(w_j|\theta_k)$$

$$y_i = \arg\max_k \sum_j t_{ij}\theta_{jk}, \text{ with } :\theta_{jk} = \log p(w_j|\theta_k) \text{ and uniform prior}$$

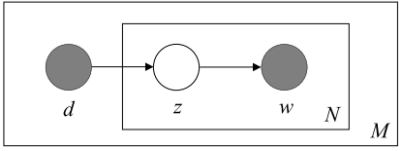
Algorithm:

Init. Random or expert knowledge
C/E Cluster assignment
M Parameter update (mean re-computation)

- K-means on top of LSA
- LSA acts as pre-processing (denoising, finding relevant words)
  - $\Rightarrow$ improved clustering over K-Means on raw data
  - Especially for large vocabulary

Probabilistic Latent Semantic Analysis

- Idea: CEM $\Rightarrow$ EM (more complex / finer)
- All documents belongs to all clusters... With a weight $p(z|d)$
- Graphical model: conditional independence $\Rightarrow p(w, d|z) = p(w|z)p(d|z)$



- Doc d is drawn from $P(d)$
- Topic z is drawn from $P(z|d)$
- Word w is drawn from $P(w|z)$

We estimate the following parameters:

- $p(d)$
- $p(\alpha|d)$
- $p(w|\alpha)$

Maximizing the log-likelyhood:

$$\mathcal{L} = \sum_{d=1}^{D} \sum_{w=1}^{W} n(d, w) \log P(d, w)$$

- Expectation (probability of the missing variables)
- Maximization

Maximizing the log-likelyhood:

$$\mathcal{L} = \sum_{d=1}^{D} \sum_{w=1}^{W} n(d, w) \log P(d, w)$$

- Expectation (probability of the missing variables)

$$P(\alpha|d, w) = \frac{P(d)P(\alpha|d)P(w|\alpha)}{\sum_{\alpha' \in \mathcal{A}} P(d)P(\alpha'|d)P(w|\alpha')}$$

- Maximization

Maximizing the log-likelyhood:

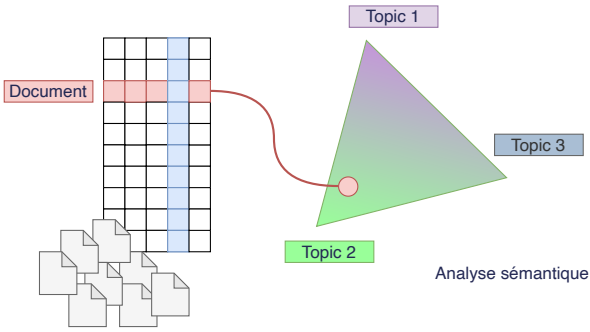
$$\mathcal{L} = \sum_{d=1}^{D} \sum_{w=1}^{W} n(d, w) \log P(d, w)$$

- Expectation (probability of the missing variables)
- Maximization

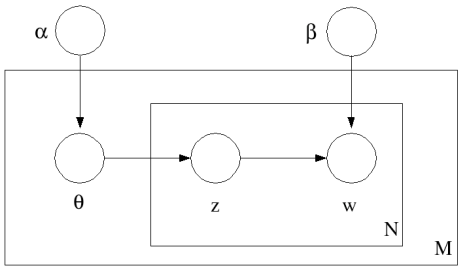
$$P(d) = \frac{\sum_{w \in \mathcal{W}} n(d, w)}{\sum_{d' \in \mathcal{D}} \sum_{w \in \mathcal{W}} n(d', w)}$$

$$P(\alpha|d) = \frac{\sum_{w \in \mathcal{W}} n(d, w) P(\alpha|d, w)}{\sum_{\alpha' \in \mathcal{A}} \sum_{w \in \mathcal{W}} n(d, w) P(\alpha'|d, w)}$$

$$P(w|\alpha) = \frac{\sum_{d \in \mathcal{D}} n(d, w) P(\alpha|d, w)}{\sum_{w' \in \mathcal{W}} \sum_{d \in \mathcal{D}} n(d, w') P(\alpha|d, w')}$$


Document
Topic 1
Topic 2
Topic 3
Analyse sémantique

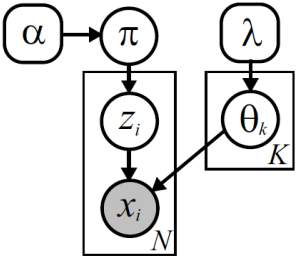
Latent Dirichlet Allocation:

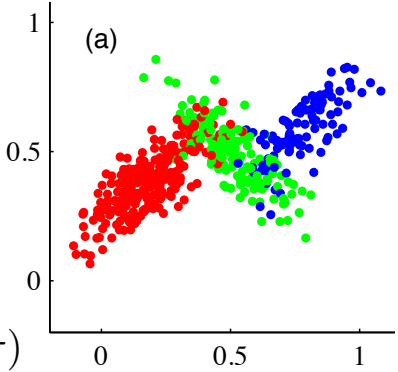


- Idea: adding a prior on the topic distribution
  - A document is supposed to belong to a topic **strongly or not**
- Learning through Gibbs sampling ($\sim$ MCMC)

not to be confused: LDA: Latent Dirichlet Allocation *vs* Linear Discriminant Analysis

On an example:





$$\theta_k = \{\mu_k, \Sigma_k\}$$

$$p(z_i \mid \pi) = \mathrm{Cat}(z_i \mid \pi)$$

$$p(x_i \mid z_i, \mu, \Sigma) = \mathcal{N}(x_i \mid \mu_{z_i}, \Sigma_{z_i})$$

Given mixture weights $\pi^{(t-1)}$ and cluster parameters $\{\theta_k^{(t-1)}\}_{k=1}^K$ from the previous iteration, sample a new set of mixture parameters as follows:

1. Independently assign each of the N data points x_i to one of the K clusters by sampling the indicator variables $z = \{z_i\}_{i=1}^N$ from the following multinomial distributions:

$$z_i^{(t)} \sim \frac{1}{Z_i} \sum_{k=1}^{K} \pi_k^{(t-1)} f(x_i \mid \theta_k^{(t-1)})\, \delta(z_i, k) \qquad Z_i = \sum_{k=1}^{K} \pi_k^{(t-1)} f(x_i \mid \theta_k^{(t-1)})$$

2. Sample new mixture weights according to the following Dirichlet distribution:

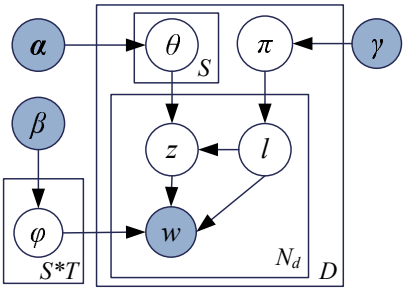
$$\pi^{(t)} \sim \mathrm{Dir}(N_1 + \alpha/K, \ldots, N_K + \alpha/K) \qquad N_k = \sum_{i=1}^{N} \delta(z_i^{(t)}, k)$$

3. For each of the K clusters, independently sample new parameters from the conditional distribution implied by those observations currently assigned to that cluster:

$$\theta_k^{(t)} \sim p(\theta_k \mid \{x_i \mid z_i^{(t)} = k\}, \lambda)$$

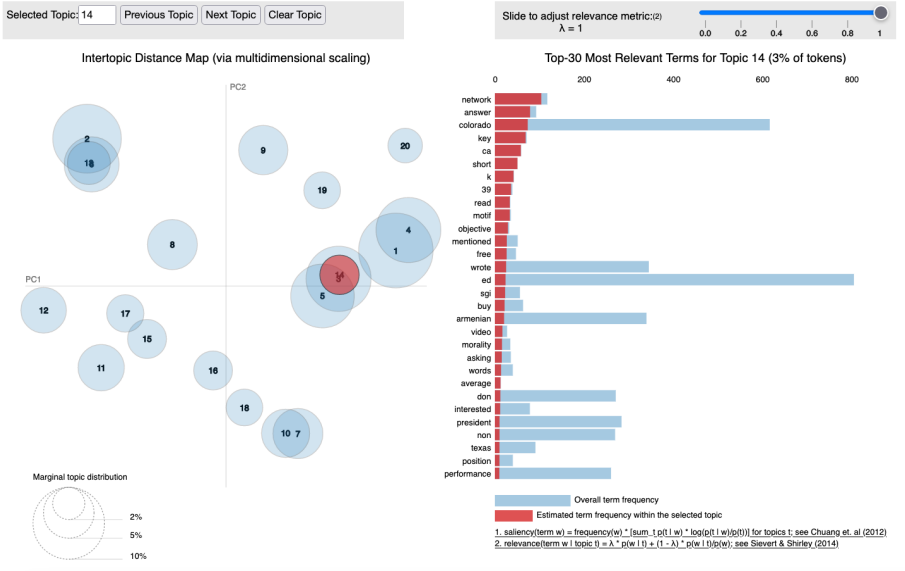
When λ defines a conjugate prior, this posterior distribution is given by Prop. 2.1.4.

- Graphical models = easy to adapt



- For each document d, choose a distribution $\pi_d \sim \text{Dir}(\gamma)$.
- For each sentiment label l under document d, choose a distribution $\theta_{d,l} \sim \text{Dir}(\alpha)$.
- For each word w_i in document d
  - choose a sentiment label $l_i \sim \text{Mult}(\pi_d)$,
  - choose a topic $z_i \sim \text{Mult}(\theta_{d,l_i})$,
  - choose a word w_i from $\varphi^{l_i}_{z_i}$, a Multinomial distribution over words conditioned on topic z_i and sentiment label l_i.

Practical: LDA viz



Conclusion on statistical semantic analysis

1. Quantitative results
   - Clustering
   - Major issue with frequent words
   - Human required in the loop (init., cluster selection, etc...)
   - Evaluation issue (purity, perplexity, ...)
2. Qualitative analysis
   - Word similarity
   - Lexical field extraction